

Construye y despliega tu primer backend profesional

Gestor de Hábitos · Una API segura, viva en internet

Módulo 3 · Clase 4 — Programa JavaScript AI Backend Developer · Dev Senior · Guía del estudiante (proyecto para replicar)

🎯 QUÉ ES ESTE DOCUMENTO

En clase construyes, junto a tu profesora, un **Gestor de Tareas**. Aquí vas a **replicar tú solo** lo mismo, pero en un dominio distinto: un **Gestor de Hábitos**. Los conceptos son idénticos; solo cambia de qué trata la app. Si lo logras en otro dominio, es la prueba de que de verdad lo entendiste.

Durante tres clases construiste una API que sabe quién eres, qué puedes hacer y que rechaza datos malos. Pero todavía vive solo en tu computador. En este reto le enseñas lo último que le falta para ser real: **salir al mundo**. Al terminar tendrás una API pública, con su propia dirección en internet, que le puedes pasar a cualquiera.

💡 CÓMO USAR ESTA GUÍA

No es un archivo para copiar y pegar. Es un mapa para que **tú construyas** la aplicación. En cada parte vas a encontrar:

- 💡 **Concepto**: qué es y con qué palabras técnicas se nombra.
- 🤔 **¿Por qué?**: la razón por la que se hace así, no de memoria.
- 🎯 **Lógica de negocio**: qué debe hacer esa pieza, exactamente.
- 🤖 **Prompt para la IA**: lo que le pides a Cursor para que te ayude a construirla.
- 💻 **En la terminal**: los comandos que vas a ejecutar.
- ✅ **Revísalo**: cómo saber que quedó bien antes de seguir.

⚠️ LA REGLA DE ORO

La IA es tu **copiloto, no tu piloto**. Te ayuda a escribir, pero **tú** decides y revisas. Nunca aceptes código que no puedas explicar. Si la IA se queda sin cupo o se equivoca, no la persigas: apóyate en esta guía y en lo que ya entiendes.

Parte 0 · Qué vas a lograr

El reto

Tu API corre con `npm run start:dev` y funciona perfecto... en tu máquina. Si le pasas `http://localhost:3000` a alguien más, no verá nada: `localhost` es *su* propio computador, no el tuyo. Aquí resolvemos eso.

El producto que vas a entregar

Este es el **componente integrador** del módulo: una API backend profesional, segura y **desplegada en la nube**. Debe incluir:

Requisito	Qué significa
Login y registro funcional	Las personas pueden crear cuenta e iniciar sesión.
Autenticación con JWT	La sesión se demuestra con un token firmado.
Roles y permisos	Un administrador puede más que un usuario normal.
Validaciones	La API rechaza datos mal formados con mensajes claros.
Swagger	Documentación interactiva en <code>/docs</code> .
Docker base	La app empaquetada en un contenedor.
Deploy en producción	Corriendo en una URL pública real.
Código en GitHub	El repositorio, sin secretos.

PARA PENSAR ANTES DE EMPEZAR

¿Qué le falta exactamente a tu API, que ya funciona en tu máquina, para que alguien en otro país la use desde su navegador? Escríbelo en una frase. Al final de la guía, compáralo con lo que aprendiste.

Parte 1 · La lógica de negocio del Gestor de Hábitos

Antes de tocar una sola línea, hay que tener clarísimo **qué** se construye. Esta es la especificación que vas a implementar. Léela completa: es tu contrato.

De qué trata la app

Una API donde cada persona se registra, inicia sesión y administra **sus propios** hábitos (por ejemplo, "meditar 10 minutos" o "leer 20 páginas"). Un **administrador** puede ver los hábitos de todos, para dar soporte. Es sencilla a propósito: el dominio no es lo difícil; el reto es sacarla a producción.

Los actores

Actor	Qué puede hacer
Visitante (sin cuenta)	Solo registrarse o iniciar sesión. Nada más.
Usuario (rol <code>USUARIO</code>)	Crear, ver, editar y borrar sus hábitos.
Administrador (rol <code>ADMIN</code>)	Todo lo del usuario, y además ver los hábitos de todos .

Las dos entidades (el modelo de datos)

🎯 ENTIDAD: USUARIO

- **id**: número, autogenerado, identifica al usuario.
- **nombre**: texto.
- **email**: texto, **único** (no se puede repetir).
- **password**: texto, pero se guarda **cifrada**, nunca tal cual.
- **rol**: `USUARIO` o `ADMIN`. **Por defecto siempre `USUARIO`**.
- **creadoEn**: fecha, automática.

🎯 ENTIDAD: HÁBITO

- **id**: número, autogenerado.
- **nombre**: texto, obligatorio (entre 3 y 120 caracteres).
- **descripcion**: texto, opcional (máximo 500 caracteres).
- **estado**: `ACTIVO`, `PAUSADO` o `ARCHIVADO`. Por defecto `ACTIVO`.
- **frecuencia**: `DIARIA`, `SEMANAL` o `MENSUAL`. Por defecto `DIARIA`.
- **usuarioid**: a qué usuario pertenece (su dueño).
- **creadoEn**: fecha, automática.

Las reglas de negocio (léelas con cuidado)

1. **Registro:** si el email ya existe, se rechaza. La contraseña se cifra antes de guardar. El usuario nace con rol `USUARIO`.
2. **El rol NO se elige al registrarse:** nadie puede pedir ser `ADMIN` desde el formulario. El ascenso se hace por fuera (a mano en la base de datos). Es una regla de seguridad, no un descuido.
3. **Login:** se verifica la contraseña; si es correcta, se entrega un token firmado que lleva adentro el rol. El mensaje de error nunca dice si el email existe o no.
4. **Propiedad (ownership):** cada usuario solo ve y toca **sus** hábitos. Pedir un hábito ajeno se rechaza.
5. **Permiso por rol:** el listado "ver los hábitos de todos" solo lo puede usar un `ADMIN`.
6. **Datos válidos:** un nombre muy corto, una frecuencia inventada, etc., se rechazan con un mensaje claro.

Los endpoints que vas a construir

Método y ruta	Qué hace	Quién puede
POST <code>/auth/register</code>	Crear cuenta	Cualquiera
POST <code>/auth/login</code>	Iniciar sesión, devuelve token	Cualquiera
POST <code>/habitos</code>	Crear un hábito propio	Con sesión
GET <code>/habitos</code>	Listar mis hábitos	Con sesión
GET <code>/habitos/:id</code>	Ver un hábito propio	Con sesión (dueño)
PATCH <code>/habitos/:id</code>	Editar un hábito propio	Con sesión (dueño)
DELETE <code>/habitos/:id</code>	Borrar un hábito propio	Con sesión (dueño)
GET <code>/habitos/admin/todos</code>	Ver los hábitos de todos	Solo <code>ADMIN</code>

💡 CÓDIGOS DE RESPUESTA QUE VAS A USAR

Toda la web habla con estos números: **200** (todo bien), **201** (creado), **400** (datos malos), **401** (no autenticado, falta token), **403** (autenticado pero sin permiso), **404** (no existe), **409** (conflicto, p. ej. email repetido). Cuando pruebes, vas a ir viendo cada uno.

Parte 2 · Preparar el terreno

Vamos a crear el proyecto y a instalar las herramientas. Aquí todavía no hay lógica: es alistar la mesa antes de cocinar.



CONCEPTO: LAS HERRAMIENTAS

- **NestJS**: el framework de Node con el que organizamos el backend en piezas ordenadas.
- **Prisma**: el traductor entre tu código y la base de datos (un ORM). Tú escribes objetos; él escribe el SQL.
- **JWT + bcrypt**: para sesiones seguras y contraseñas cifradas.
- **class-validator**: para validar lo que llega.
- **Swagger**: para documentar la API sola.

Paso 2.1 — Crear el proyecto



EN LA TERMINAL (EN TU CARPETA DE PROYECTOS)

```
npm @nestjs/cli@11 new gestor-habitos --package-manager npm
```



OJO: FIJA LA VERSIÓN 11

Escribe `@nestjs/cli@11`, no lo dejes sin número. La versión más nueva usa otra configuración que rompe lo que vamos a hacer. Fijar la versión te ahorra horas de dolor.

Paso 2.2 — Instalar el stack

Entra al proyecto (`cd gestor-habitos`) y luego:



EN LA TERMINAL (DENTRO DE GESTOR-HABITOS)

```
npm install @nestjs/jwt@11 @nestjs/passport@11 passport passport-jwt bcrypt class-validator class-transformer @nestjs/swagger@11 helmet dotenv @prisma/client@7 @prisma/adapter-pg pg
npm install -D prisma@7 @types/passport-jwt @types/bcrypt @types/pg
```



REVÍSALO

Corre `npm run start:dev`. Debe salir **"Nest application successfully started"**. Si arranca, la mesa está lista. Detén el servidor con `Ctrl + C`.

Parte 3 · La estructura de archivos

💡 CONCEPTO: ARQUITECTURA POR CAPAS

NestJS organiza cada funcionalidad en piezas con un solo trabajo cada una:

- **Módulo:** junta las piezas de una funcionalidad (ej. `auth`, `habitos`).
- **Controller:** recibe la petición HTTP y responde.
- **Service:** la lógica de negocio (las reglas).
- **DTO:** define y valida la forma de los datos que entran.

😞 ¿POR QUÉ SEPARAR ASÍ?

Para que cada archivo sea fácil de entender, cambiar y probar. Si todo estuviera en un solo archivo gigante, un cambio pequeño podría romper todo. Es el principio de **una responsabilidad por pieza**.

Paso 3.1 — Crear los archivos vacíos con IA

No le pidas a la IA que escriba el código: solo que cree los archivos vacíos (es rápido y no gasta tu cupo). El contenido lo construyes tú en las partes siguientes.

🤖 PROMPT PARA CURSOR (SOLO ESTRUCTURA)

Crea la siguiente estructura de carpetas y archivos **vacíos**. No escribas código dentro de ellos, solo crea los archivos:

```
src/prisma/prisma.service.ts, src/prisma/prisma.module.ts, src/auth/dto/register.dto.ts,
src/auth/dto/login.dto.ts, src/auth/auth.service.ts, src/auth/auth.controller.ts, src/auth/
auth.module.ts, src/auth/jwt.strategy.ts, src/auth/jwt-auth.guard.ts, src/auth/usuario-
actual.decorator.ts, src/auth/roles.decorator.ts, src/auth/roles.guard.ts, src/habitos/dto/
crear-habito.dto.ts, src/habitos/dto/actualizar-habito.dto.ts, src/habitos/
habitos.service.ts, src/habitos/habitos.controller.ts, src/habitos/habitos.module.ts, src/
common/filters/http-exception.filter.ts, .env, .env.example, Dockerfile, .dockerignore
```

💻 ALTERNATIVA POR TERMINAL (SI NO QUIERES USAR LA IA)

```
mkdir -p src/prisma src/auth/dto src/habitos/dto src/common/filters
touch src/prisma/prisma.service.ts src/prisma/prisma.module.ts src/auth/
auth.service.ts src/auth/auth.controller.ts src/auth/auth.module.ts src/auth/
jwt.strategy.ts src/auth/jwt-auth.guard.ts src/auth/usuario-actual.decorator.ts src/
auth/roles.decorator.ts src/auth/roles.guard.ts
touch src/auth/dto/register.dto.ts src/auth/dto/login.dto.ts src/habitos/
habitos.service.ts src/habitos/habitos.controller.ts src/habitos/habitos.module.ts
src/habitos/dto/crear-habito.dto.ts src/habitos/dto/actualizar-habito.dto.ts src/
common/filters/http-exception.filter.ts .env .env.example Dockerfile .dockerignore
```

En Windows, si `touch` no existe, crea los archivos a mano en el explorador de Cursor.



`src/main.ts`, `src/app.module.ts` y `prisma/schema.prisma` ya existen (los crearon las herramientas). Esos se **editan**, no se crean.

Parte 4 · El modelo de datos (Prisma)

💡 CONCEPTO

El **schema** de Prisma es el plano de tu base de datos: describe las tablas y sus columnas en un solo archivo. Una **migración** es el paso que convierte ese plano en tablas reales. El **cliente** de Prisma es el código que usas para consultar esas tablas desde la app.

🤔 ¿POR QUÉ UN ORM Y NO SQL A MANO?

Porque escribes en el lenguaje de tu app (objetos), no en SQL suelto, y Prisma se encarga de traducir. Menos errores, y si cambias algo, la migración lo aplica de forma ordenada y repetible.

🎯 LÓGICA DE NEGOCIO (RECUERDA LA PARTE 1)

Necesitas **Usuario** (con `rol` por defecto `USUARIO`) y **Hábito** (con `estado`, `frecuencia` y su `usuarioId` dueño). El `email` es único. Un hábito pertenece a un usuario (relación uno-a-muchos).

🤖 PROMPT PARA CURSOR

En `prisma/schema.prisma`, define el modelo de datos de un gestor de hábitos usando el generador "prisma-client" con salida `../src/generated/prisma` y `moduleFormat "cjs"`. Modelos: `Usuario` (`id` autoincremental, `nombre`, `email` único, `password`, `rol` enum `USUARIO|ADMIN` con default `USUARIO`, `creadoEn`) y `Habito` (`id`, `nombre`, `descripcion` opcional, `estado` enum `ACTIVO|PAUSADO|ARCHIVADO` default `ACTIVO`, `frecuencia` enum `DIARIA|SEMANAL|MENSUAL` default `DIARIA`, `usuarioId`, `creadoEn`, relación con `Usuario`). Luego crea `src/prisma/prisma.service.ts` como un `PrismaService` que use el driver adapter `@prisma/adapter-pg` leyendo `process.env.DATABASE_URL`, y su `prisma.module.ts` global.

⚠️ REVISALO QUE PROPONE LA IA

Confirma tres cosas antes de aceptar: (1) el `rol` tiene default `USUARIO`; (2) el `email` es único; (3) el `Habito` tiene `usuarioId`. Si algo no está, pídeselo explícito.

Paso 4.1 — Crear las tablas

💻 EN LA TERMINAL

```
npx prisma migrate dev --name init
npx prisma generate
```

✓ REVÍSALO

Debe crearse la carpeta `prisma/migrations/` y aparecer el cliente en `src/generated/prisma`. Si el editor marca en rojo los `import` de Prisma, corre otra vez `npx prisma generate`. Puedes abrir `npx prisma studio` para ver tus tablas vacías.

Parte 5 · Autenticación (registro y login)

💡 CONCEPTO

Autenticación es responder "¿quién eres?". Dos piezas clave: **bcrypt** cifra la contraseña de una sola vía (nunca se guarda ni se puede leer en texto plano), y el **JWT** es un "carné" firmado que el servidor entrega al iniciar sesión y que el cliente presenta después.

🤔 ¿POR QUÉ UN TOKEN Y NO GUARDAR LA SESIÓN EN EL SERVIDOR?

Porque el token viaja con cada petición y se verifica por su firma, sin que el servidor tenga que recordar quién está conectado. Eso hace la API más simple de escalar. El token lleva adentro el `id` y el `rol`.

🔗 LÓGICA DE NEGOCIO

- **Registro:** si el email ya existe → error `409`. Si no, cifra la contraseña con `bcrypt` y crea el usuario (nace `USUARIO`). Responde sin la contraseña.
- **Login:** busca por email, compara la contraseña. Si falla, error `401` con mensaje **genérico** (no digas si el email existe). Si acierta, firma un JWT que incluye `id`, `email` y `rol`, con expiración de 1 hora.
- **Regla de oro:** el `rol` nunca llega desde el formulario de registro.

🤖 PROMPT PARA CURSOR

Construye el módulo auth en NestJS con estas reglas exactas: `register` recibe nombre, email, password; si el email ya existe lanza `409`; cifra la password con `bcrypt` (costo `10`); crea el usuario con rol `USUARIO` por defecto; nunca aceptes "rol" desde el body. `login` verifica la password con `bcrypt.compare`; si falla lanza `401` con un mensaje genérico que NO revele si el email existe; si acierta firma un JWT (con `@nestjs/jwt`) cuyo payload incluye `sub` (`id`), `email` y `rol`, con expiración `1h` leyendo el secreto de `process.env.JWT_SECRET`. Incluye `JwtStrategy` (`passport-jwt`), `JwtAuthGuard` y un decorador `@UsuarioActual` para leer el usuario del token. Sigue las reglas de `.cursor/rules` si existen.

⚠️ REVISALO QUE PROPONE LA IA

Que use `@nestjs/jwt` (no otra librería), que el mensaje de error de login sea genérico, que la contraseña NO viaje en la respuesta, y que el secreto se lea de `process.env.JWT_SECRET` y no esté escrito en el código.

Parte 6 · Autorización (roles y propiedad)

💡 CONCEPTO

Autenticación es "¿quién eres?"; **autorización** es "¿qué te dejo hacer?". Se resuelve con dos ideas: **roles** (RBAC: tu cargo define tus permisos) y **propiedad** (ownership: cada quien solo toca lo suyo). En NestJS, un **guard** es el "portero" que deja pasar o no.

🤔 ¿POR QUÉ DOS GUARDS DISTINTOS?

Porque cada uno tiene un solo trabajo: **JwtAuthGuard** revisa "¿traes token válido?" y **RolesGuard** revisa "¿tu rol alcanza?". Separarlos permite combinarlos solo donde hacen falta, sin repetir lógica. Es responsabilidad única, otra vez.

🎯 LÓGICA DE NEGOCIO

- Toda ruta de hábitos exige token (**JwtAuthGuard**). Sin token → **401** .
- Crear, listar, ver, editar y borrar hábitos: siempre **filtrando por el dueño** (**usuarioId** del token). Pedir un hábito ajeno → **403** . Un hábito que no existe → **404** .
- El endpoint **GET /habitos/admin/todos** exige además rol **ADMIN** (**RolesGuard** + **@Roles**). Un usuario normal → **403** .
- El dueño de un hábito nunca viene del body: se toma del token.

🤖 PROMPT PARA CURSOR

Construye el módulo `habitos` en NestJS con CRUD y estas reglas: todas las rutas usan `JwtAuthGuard`. El dueño (`usuarioId`) de cada hábito se toma SIEMPRE del usuario autenticado (del token), nunca del body. `crear`, `listar`, `ver`, `editar` y `borrar` operan solo sobre los hábitos del usuario actual; si un hábito no existe lanza 404, y si existe pero es de otro usuario lanza 403. Agrega un endpoint `GET /habitos/admin/todos` que liste los hábitos de todos los usuarios (incluyendo `id`, `nombre` y `email` del dueño, nunca su contraseña) y protégelo con un `RolesGuard` y un decorador `@Roles([ADMIN])` además del `JwtAuthGuard`. Crea también el decorador `@Roles` y el `RolesGuard` que lee esa metadata.

⚠️ REvisa lo que propone la IA

Que `admin/todos` tenga **los dos** guards (token y rol), que el CRUD filtre por el usuario del token, y que la respuesta del admin no incluya contraseñas. Prueba mentalmente: "¿un usuario normal podría ver el hábito de otro?" La respuesta debe ser no.

Parte 7 · Validaciones y manejo de errores

CONCEPTO

Una **validación** revisa que lo que entra cumpla las reglas (con `class-validator` en los DTO). Un **filtro de excepciones** hace que **todos** los errores salgan con la misma forma, sin filtrar detalles internos del servidor.

¿POR QUÉ UN FORMATO ÚNICO DE ERROR?

Porque quien consume tu API (un frontend, otra persona) necesita poder confiar en que un error *siempre* se ve igual. Y porque mostrar el error crudo de un bug puede filtrar información interna: es un riesgo de seguridad.

LÓGICA DE NEGOCIO

- **Registro:** nombre mínimo 2 caracteres, email con formato válido, password mínimo 8. Si algo falla → `400` con el mensaje.
- **Crear/editar hábito:** nombre entre 3 y 120 caracteres; descripción opcional (máx 500); frecuencia y estado solo aceptan sus valores permitidos.
- **Errores:** toda respuesta de error sale con `statusCode`, `timestamp`, `path` y `message`. Un bug inesperado responde `500` genérico y el detalle queda solo en los logs del servidor.

PROMPT PARA CURSOR

Agrega validaciones con `class-validator` a los DTOs: en registro, nombre con mínimo 2 caracteres, email válido y password mínimo 8; en crear hábito, nombre obligatorio entre 3 y 120 caracteres, descripción opcional máximo 500, y frecuencia restringida al enum (DIARIA, SEMANAL, MENSUAL). Todos los mensajes en español. Crea un filtro global de excepciones (`@Catch()` sin argumentos) que responda siempre con `statusCode`, `timestamp`, `path` y `message`; si el error no es una `HttpException`, responde 500 con mensaje genérico y registra el detalle solo con `console.error`. Activa el `ValidationPipe` global con `whitelist` y `transform` en `main.ts`.

REVISAR

Que los mensajes salgan en español, y que un error inesperado **no** muestre detalles internos en la respuesta (solo el `500` genérico).

Parte 8 · Documentación con Swagger

CONCEPTO

Swagger lee tu código y genera una página interactiva (en `/docs`) donde cualquiera puede ver y probar tus endpoints. Es el manual de tu API, que se actualiza solo.

LÓGICA DE NEGOCIO

Todos los endpoints documentados en `/docs`, agrupados por tema (auth, hábitos), y un botón **Authorize** para pegar el token y probar las rutas protegidas.

PROMPT PARA CURSOR

Configura Swagger (@nestjs/swagger) en main.ts: título "Gestor de Hábitos API", versión 1.0, con `addBearerAuth()` para poder autenticar desde la interfaz, y publícalo en la ruta `/docs`. Agrega `@ApiTags` y `@ApiBearerAuth` en los controladores para que las rutas queden agrupadas y muestren el candado.

REVÍSALO

Con el servidor corriendo, abre `http://localhost:3000/docs`. Deben aparecer todas tus rutas y el botón **Authorize** arriba a la derecha.

Parte 9 · Probar en tu máquina (Postman)

Antes de pensar en la nube, comprueba que todo funciona en local. Necesitas tu archivo `.env` con la conexión a tu Postgres local y un `JWT_SECRET` cualquiera (lo vemos en la Parte 10). Arranca con `npm run start:dev`.

A — Registrar un usuario

POST `http://localhost:3000/auth/register` · Body → raw → JSON:

BODY

```
{ "nombre": "Ana", "email": "ana@correo.com", "password": "clave1234" }
```

Esperado: `201` y responde tu usuario, **sin** la contraseña.

B — Iniciar sesión

POST `http://localhost:3000/auth/login` · mismo body con email y password.

Esperado: `201` y un `access_token`.

C — Guardar el token

Copia el valor de `access_token`. En la siguiente petición ve a la pestaña **Authorization** → tipo **Bearer Token** → pégalo.

D — Crear un hábito

POST `http://localhost:3000/habitos` (con el Bearer Token) · Body → raw → JSON:

BODY

```
{ "nombre": "Meditar 10 minutos", "frecuencia": "DIARIA" }
```

Esperado: `201` con el hábito creado (estado `ACTIVO`).

E — Listar mis hábitos

GET `http://localhost:3000/habitos` (con el token).

Esperado: `200` y una lista con tu hábito.

✓ SI ESTOS 5 PASOS FUNCIONAN, TU BACKEND ESTÁ LISTO PARA DESPLEGAR

Todo lo que sigue es sacarlo de tu máquina.

🧠 COMPRUEBA QUE LA SEGURIDAD ESTÁ VIVA (30 SEGUNDOS)

GET `/habitos` **sin** token → debe dar `401`. Y POST `/habitos` con body `{ "nombre": "ab" }` → debe dar `400`. Si esos dos fallan como deben fallar, tu API está protegida.

Parte 10 · Variables de entorno

💡 CONCEPTO

Una **variable de entorno** es un dato de configuración que vive **fuera** del código, en el lugar donde la app se ejecuta, y que el programa lee al arrancar (en Node, con `process.env`).

🤔 ¿POR QUÉ SACAR LA CONFIGURACIÓN DEL CÓDIGO?

Por dos razones. **Seguridad:** si tu secreto para firmar tokens está escrito en el código y subes eso a GitHub, cualquiera podría firmar tokens falsos y hacerse pasar por otros. **Portabilidad:** la base de datos y el secreto son distintos en tu máquina y en la nube; si están en el código, tendrías que editarlo para cada lugar. Analogía: una cafetera funciona en cualquier país, pero el voltaje se lo da el enchufe, no viene soldado.

🔗 LÓGICA DE NEGOCIO: TUS TRES VARIABLES

Variable	Para qué	Cambia según
<code>DATABASE_URL</code>	Conexión a la base	Local vs. nube
<code>JWT_SECRET</code>	Firmar los tokens	Secreto de cada entorno
<code>PORT</code>	Puerto donde escucha	Lo pone la nube sola

El código no cambia entre local y producción: solo cambia el **valor** de estas variables.

💡 TRES ARCHIVOS QUE DEBES DISTINGUIR

- `.env` → tus valores reales, con secretos. **NUNCA se sube a GitHub.**
- `.env.example` → un ejemplo con los nombres de las variables pero sin valores reales. Este **sí** se sube, para que otros sepan qué configurar.
- `.gitignore` → la lista de lo que Git ignora. Debe incluir `.env`.

🤖 PROMPT PARA CURSOR

Crea un archivo `.env.example` con las variables `DATABASE_URL` (con un valor de ejemplo del Session Pooler de Supabase usando placeholders), `JWT_SECRET` (con instrucción de poner uno largo y aleatorio) y `PORT=3000`, cada una con un comentario que explique para qué sirve. Asegúrate de que `.gitignore` incluya la línea `.env` para que el archivo con secretos nunca se suba.

OJO — EL ERROR MÁS COMÚN DEL DÍA

Confirma que `.env` está en `.gitignore` **antes** del primer `git push`. Si un secreto llega a subirse, queda en el historial aunque lo borres después. La regla: **el `.env` nunca sale de tu máquina.**

Parte 11 · Docker

CONCEPTO

Docker empaqueta tu app con todo lo que necesita (Node, librerías, tu código) en una **imagen**, y la ejecuta como un **contenedor** que corre igual en cualquier máquina. La **imagen** es la plantilla; el **contenedor** es esa plantilla ya corriendo.

¿POR QUÉ?

Para acabar con el "en mi máquina funciona". El servidor de la nube tiene otro sistema y otras versiones; si empaquetas tu app en un contenedor, corre idéntica allá que acá. Analogía: un contenedor de barco: cualquier grúa lo maneja igual, sin importar qué lleve adentro.

INSTALAR DOCKER ES OPCIONAL

Para desplegar **no** necesitas instalar Docker en tu máquina. El **Dockerfile** es solo un archivo de texto; Render (la nube) lo construye y lo corre por ti. Solo instalarías Docker Desktop si quieres probar el contenedor en tu propio computador, que es un extra, no un requisito.

QUÉ DEBE HACER TU DOCKERFILE

Es la receta de la imagen: partir de una máquina con Node, instalar dependencias, generar Prisma, compilar el proyecto y, al arrancar, **aplicar las migraciones y luego prender el servidor**. Ese "aplicar migraciones al arrancar" es lo que crea las tablas en la base de producción la primera vez.

PROMPT PARA CURSOR

Crea un Dockerfile para esta app NestJS: base node:22-slim, instala openssl (Prisma lo usa), copia package.json e instala con npm ci, copia el resto, corre "npx prisma generate" y "npm run build", y en el CMD ejecuta primero "npx prisma migrate deploy" y luego "node dist/src/main.js". Crea también un .dockerignore que excluya node_modules, dist, .git y .env.

REVISAR

Que el **.dockerignore** excluya el **.env**: los secretos **nunca** entran a la imagen. En producción los entrega Render, no el archivo.

Parte 12 · La base de datos en la nube (Supabase)

💡 CONCEPTO

Supabase te da un PostgreSQL alojado en internet, gratis para lo nuestro. Tu Postgres local vive en tu máquina apagada; la nube necesita una base que también esté en internet.

🎯 PASOS

1. Crea cuenta en supabase.com y un **New project**.
2. Elige nombre y una **contraseña de base de datos** (guárdala: es parte de tu cadena de conexión).
3. Espera un par de minutos a que se cree.
4. Botón **Connect** → copia la cadena del **Session Pooler**.

⚠️ OJO — EL SEGUNDO ERROR MÁS COMÚN

Supabase ofrece varias conexiones. Usa la del **Session Pooler** (dice pooler.supabase.com). **No** uses la conexión "directa" (db...supabase.co): esa es IPv6 y Render no la alcanza, así que tu app quedaría desplegada pero **sin poder hablar con la base**. Regla simple: la cadena que dice pooler.

😞 ¿POR QUÉ LA MISMA VARIABLE SIRVE EN LOCAL Y EN LA NUBE?

Porque tu código solo lee `process.env.DATABASE_URL`; no le importa el valor. En tu máquina apunta a tu Postgres; en Render apuntará a Supabase. Cambiar de base es cambiar un valor de entorno, no el código.

Parte 13 · Subir el código a GitHub

CONCEPTO

GitHub guarda tu código en la nube. Y es parte del deploy: Render no recibe un archivo comprimido, se **conecta a tu repositorio** y, cada vez que subes cambios, vuelve a desplegar.

ANTES DE SUBIR, VERIFICA

- `.env` **está** en `.gitignore` (no se sube).
- `node_modules` y `dist` **están** ignorados.
- `prisma/migrations/` **NO** está ignorada (sí se sube: la nube la necesita para crear las tablas).

EN LA TERMINAL (EN LA RAÍZ DEL PROYECTO)

```
git init
git add .
git commit -m "gestor-habitos: backend seguro listo para deploy"
```

Luego crea un repositorio **vacío** en GitHub (sin README) y conéctalo:

EN LA TERMINAL

```
git remote add origin https://github.com/TU-USUARIO/gestor-habitos.git
git branch -M main
git push -u origin main
```

REVÍSALO CON TUS PROPIOS OJOS

Entra al repositorio en el navegador y confirma que **NO** aparece el archivo `.env`. Si aparece, detente y arregla el `.gitignore` antes de seguir.

Parte 14 · Deploy en Render

💡 CONCEPTO

Render es una plataforma que toma tu repositorio, construye la imagen con tu `Dockerfile`, la ejecuta en un servidor siempre encendido y le da una **URL pública con HTTPS**. Aquí se junta todo: GitHub + Docker + variables de entorno + Supabase.

🎯 PASOS

1. En `render.com` : **New** → **Web Service**.
2. Conecta tu GitHub y elige el repo `gestor-habitos`.
3. Render detecta el `Dockerfile` (Runtime: Docker). Instance Type: **Free**.
4. **Antes de desplegar**, agrega las variables de entorno (ver abajo).
5. Dale **Create Web Service** y observa los **Logs**.

🎯 VARIABLES DE ENTORNO EN RENDER (AQUÍ "ATERRIZA" LA PARTE 10)

Key	Value
<code>DATABASE_URL</code>	la cadena del Session Pooler de Supabase
<code>JWT_SECRET</code>	un secreto largo y aleatorio (no el de pruebas)

No configures `PORT` : Render la inyecta sola y tu app ya la lee.

😞 ¿VES LO QUE PASÓ CON EL SECRETO?

Terminó **aquí**, en la configuración del servidor, no en el código. Por eso desde el principio lo sacamos del código. Código en GitHub (compartible) + configuración privada en Render = la separación que estudiaste en la Parte 10, ahora real.

⚠️ PLAN GRATIS: EL SERVICIO "SE DUERME"

Tras un rato sin uso, la primera petición tarda ~30-50 segundos en despertar. No es un error; las siguientes van rápidas.

Al terminar, arriba aparece tu **URL pública**, algo como `https://gestor-habitos.onrender.com`.

Parte 15 · Probar en producción

QUÉ HACER

Repite las pruebas de la Parte 9, pero cambiando `http://localhost:3000` por tu URL de Render. Abre primero `https://TU-APP.onrender.com/docs` : es el mismo Swagger, ahora público.

- Registro y login contra la URL pública → funcionan.
- `GET /habitos` sin token → `401` .
- `GET /habitos/admin/todos` con usuario normal → `403` .

PARA PENSAR

Si asciendes un usuario a `ADMIN` en Supabase y aun así `/habitos/admin/todos` le da `403` , ¿qué pasó? Pista: el rol viaja **dentro** del token, congelado al momento del login. Solución: volver a iniciar sesión para recibir un token nuevo con el rol actualizado.

Parte 16 · Tu entregable y cómo presentarlo

📋 CHECKLIST FINAL

- ☐ Login y registro funcionan en la URL pública
- ☐ JWT: el login devuelve token y las rutas protegidas lo exigen
- ☐ Roles: `/habitos/admin/todos` solo funciona con rol ADMIN
- ☐ Validaciones: datos malos devuelven 400 con mensaje
- ☐ Swagger disponible en `/docs`
- ☐ Dockerfile en el repositorio
- ☐ Desplegado en Render con URL pública
- ☐ Código en GitHub, **sin** el `.env`

💡 CÓMO PRESENTAR (MUESTRA 3 COSAS)

1. La **URL pública** funcionando: entra a `/docs`.
2. Una prueba en vivo: registro → login → crear un hábito.
3. El **repositorio** en GitHub, señalando que el `.env` no está ahí.

Con eso demuestras las tres cosas que se evalúan: **funciona**, es **seguro** y está **bien construido**.

Glosario rápido

Término	En una frase
API	Un programa que responde peticiones de otros programas.
Endpoint	Una dirección concreta de la API (ruta + método) que hace algo.
ORM (Prisma)	Traductor entre tu código y la base de datos.
Migración	El paso que convierte tu modelo en tablas reales.
bcrypt	Cifra contraseñas de una sola vía.
JWT	Carné firmado que prueba tu sesión en cada petición.
Guard	Portero que deja pasar o no una petición.
Ownership	Regla de que cada quien solo toca lo suyo.
Variable de entorno	Configuración que vive fuera del código.
Contenedor (Docker)	Tu app empaquetada para correr igual en cualquier lado.
Deploy	Publicar tu app en un servidor con URL pública.
Session Pooler	La conexión (IPv4) de Supabase que la nube sí alcanza.

Errores comunes (y cómo leerlos)

Síntoma	Causa probable	Solución
Render "Live" pero error de base	Usaste la conexión directa (IPv6) de Supabase	Cambia a la cadena del Session Pooler
El deploy no queda "Live"	La app escucha en un puerto fijo	Debe leer <code>process.env.PORT</code>
<code>.env</code> aparece en GitHub	Faltó ponerlo en <code>.gitignore</code>	Ignóralo, quítalo del repo y cambia el secreto expuesto
403 aunque el usuario ya es admin	El token viejo aún dice USUARIO	Vuelve a iniciar sesión
Primera petición muy lenta	El plan gratis "durmió" el servicio	No es error; espera a que despierte

CIERRE

Vuelve a la frase que escribiste al inicio. Lograste algo real: tomaste los mismos conceptos de la clase y los llevaste a un dominio nuevo, tú solo. Separaste la configuración del código, empaquetaste la app con Docker y la publicaste en internet con su base de datos en la nube. Eso es, exactamente, llevar software a producción.